ISTANBUL TECHNICAL UNIVERSITY

COMPUTER ENGINEERING DEPARTMENT

BLG 222E

COMPUTER ORGANIZATION
PROJECT 1 REPORT

CRN : 23728

LECTURER : Mustafa Ersel Kamaşak

GROUP MEMBERS:

150240928 : Ravan Seyfullayev

150230932 : Raimbek Erik

SPRING 2026

Contents

FRONT COVER **1**

CONTENTS **1**

1 INTRODUCTION **1**

2 CONTRIBUTION **1**

3 MATERIALS AND METHODS **2**

3.1 Part 1: 16-bit Register Design 2

3.2 Part 2: Register Files (RF and ARF) 3

3.3 Part 3: Memory Units (DMU and IMU) 6

3.4 Part 4: ALU and System Integration 7

4 RESULTS **9**

4.1 Register and Register File Performance 10

4.2 Memory Unit Synchronization 10

4.3 ALU and Top-Level Integration 10

5 DISCUSSION **12**

6 CHALLENGES AND TROUBLESHOOTING **12**

6.1 ALU Combinational vs. Sequential Logic 12

6.2 Register File (RF) Logic and Bit Ordering 12

6.3 Address Register File (ARF) Discrepancy 13

6.4 Data Memory (DM) Conflict 13

6.5 Simulation Configuration: File Namings and Timescales 13

7 CONCLUSION **13**

REFERENCES **14**

1 INTRODUCTION

In this project, we designed and implemented the core components of a modular 16-bit computer organization using Verilog HDL [1]. The scope of the work involved creating fundamental building blocks, specifically a versatile 16-bit register, and scaling these components into complex subsystems including a General Purpose Register File (RF), an Address Register File (ARF), and specialized Memory Units (DMU and IMU). The final phase focused on constructing a 16-operation Arithmetic Logic Unit (ALU) with integrated flag management. The primary goal was to ensure synchronous operation across all modules and verify the data flow through a top-level organization utilizing a centralized multiplexer system.

2 CONTRIBUTION

This section outlines individual responsibilities in the project.

Table 1: Task Distribution and Responsibilities

Member	Role	Assigned Tasks
Ravan Seyfullayev	Hardware Design & Integration	• Implementation of 16-bit Register logic. • Design of ALU operations and Flag management.
Raimbek Erik	Memory Units & Verification	• Development of DMU and IMU byte-loading logic. • Testbench execution and simulation verification.
Both Members	Documentation & Troubleshooting	• Debugging ARF and DMU logic issues. • Technical report writing and RTL. • Design of ALU system. Top-level module integration and MUX routing.

3 MATERIALS AND METHODS

The design was implemented using Verilog HDL and verified using the Vivado 2017.4 design suite [2]. The system architecture is divided into four main functional parts.

3.1 Part 1: 16-bit Register Design

As seen from Figure 1, the core building block is a 16-bit register that transforms its state Q to Q^+ based on a 2-bit FunSel bus. When the master Enable (E) is low (0), the register enters a Retain state, ignoring all other inputs. Upon a high enable signal (1), the 00 code triggers a Clear (forcing Q to 0), while 01 executes a Load, mapping the 16-bit input bus I directly to the register. Arithmetic capabilities are defined by 10 for Increment ($Q + 1$) and 11 for Decrement ($Q - 1$), allowing the register to function as a

E	FunSel	Q^+
0	ϕ	Q (Retain value)
1	00	0 (Clear)
1	01	I (Load)
1	10	$Q + 1$ (Increment)
1	11	$Q - 1$ (Decrement)

Table 2: Characteristic table of 16 bit register.

counter [3].

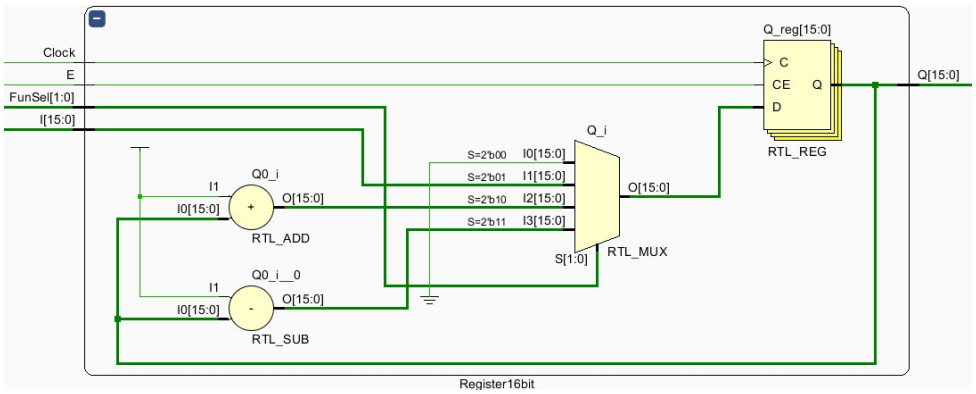

Figure 1: Inputs and Outputs of the 16 bit register module

3.2 Part 2: Register Files (RF and ARF)

We developed two distinct register management units:

- **Register File (RF):** As seen in Figure 2, this unit integrates eight registers. The RegSel (4-bit) and ScrSel (4-bit) signals act as decoders that determine which combination of the four general-purpose registers (R1-R4) and four scratch registers (S1-S4) receive the operation defined by FunSel. For data retrieval, two 16-bit asynchronous ports, OutA and OutB, are used. Their contents are determined by OutASel and OutBSel (3-bit), which act as multiplexer addresses to select one of the eight internal registers for output.

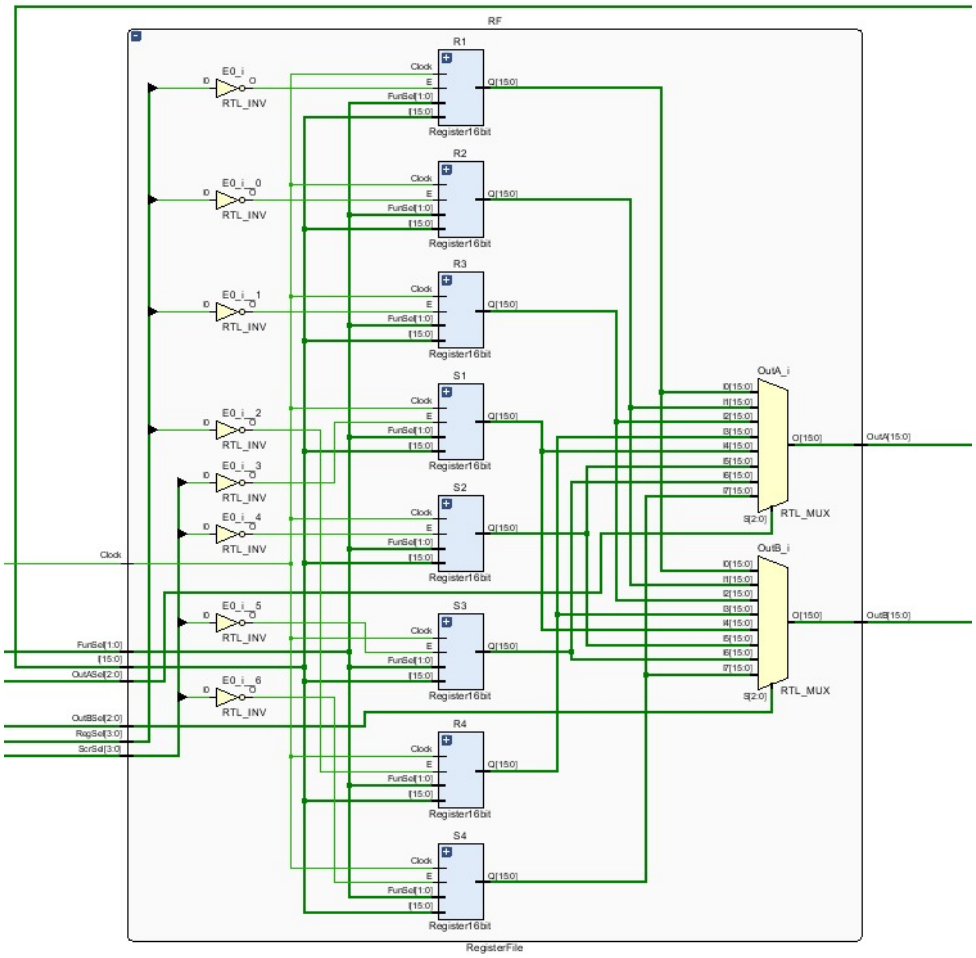

Figure 2: Inputs and Outputs of the RF module

- **Address Register File (ARF):** As shown in Figure 3, this unit houses the Program Counter (PC), Stack Pointer (SP), and Address Register (AR). Selection for modification is handled by a 3-bit RegSel which can enable multiple registers simultaneously. The output logic features three ports: OutC and OutD are multiplexed via OutCSel (2-bit) and OutDSel (1-bit) respectively, while OutE serves as a dedicated, transparent window constantly broadcasting the PC value to the rest of the system.

OutCSel	**Out**
00	PC
01	PC
10	AR
11	SP

OutDSel	OutD
0	AR
1	SP

Table 3: OutCSel and OutDSel control

RegSel	Enable General Purpose Registers	RegSel	Enable General Purpose Registers
000	All registers are enabled. (Function selected by FunSel will be applied to PC, SP, and AR.)	**100**	SP and AR are enabled. (Function selected by FunSel will be applied to SP and AR.)
001	PC and SP are enabled. (Function selected by FunSel will be applied to PC and SP.)	**101**	Only SP is enabled. (Function selected by FunSel will be applied to SP.)
010	PC and AR are enabled. (Function selected by FunSel will be applied to PC.)	**110**	Only AR is enabled. (Function selected by FunSel will be applied to AR.)
011	Only PC is enabled. (Function selected by FunSel will be only applied to PC.)	**111**	NO registers are enabled. (All registers retain their values.)

Table 4: Regsel control input table

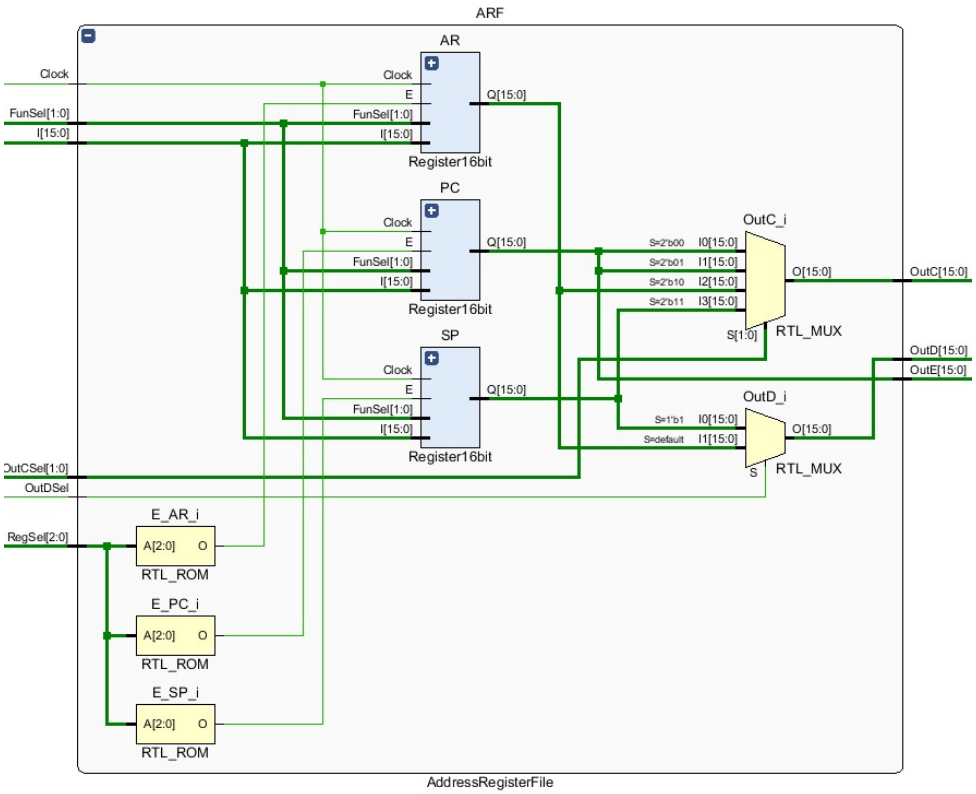

Figure 3: Inputs and Outputs of the ARF module

3.3 Part 3: Memory Units (DMU and IMU)

To address the 8-bit bus bottleneck, we implemented:

- **Data Memory Unit (DMU):** As seen in Figure 4, this unit manages a RAM module and a 16-bit Data Register (DR). The CS (Chip Select) signal enables the unit at logic 1. The WR signal dictates the RAM's read/write mode. To load the DR, FunSel is used as a byte-stepper: code 0 directs the 8-bit input I to the lower byte (DR[7:0]), and code 1 directs it to the upper byte (DR[15:8]).

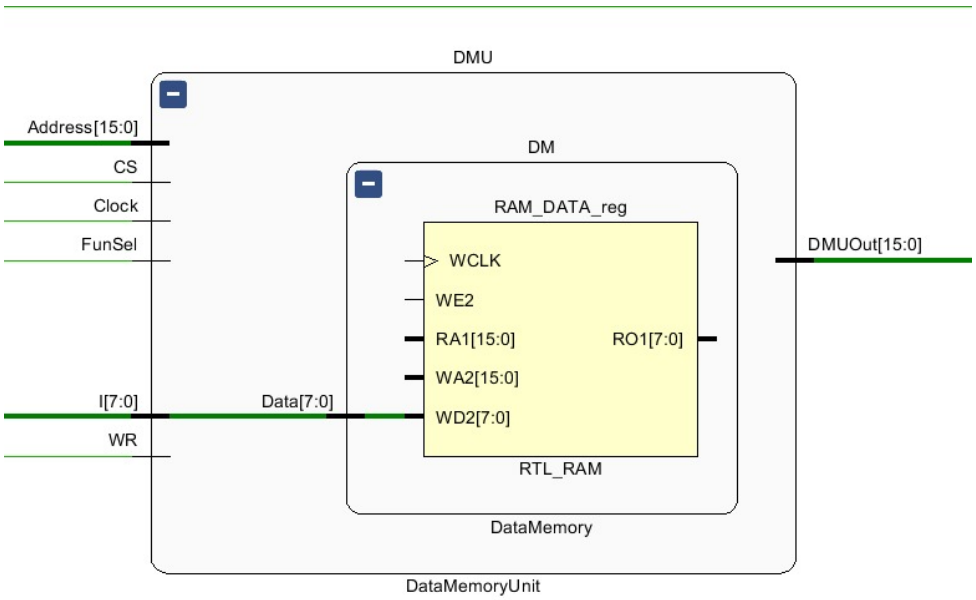

Figure 4: Inputs and Outputs of the DMU module

- **Instruction Memory Unit (IMU):** As shown in Figure 5, this unit pairs a ROM with an Instruction Register (IR). It relies on an L'H signal; a 0 value loads the 8-bit bus into the lower half of the IR, while a 1 loads the upper half. The unit provides two outputs: the full 16-bit IROut and a padded 16-bit IMUOut which carries only the 8 LSBs of the instruction.

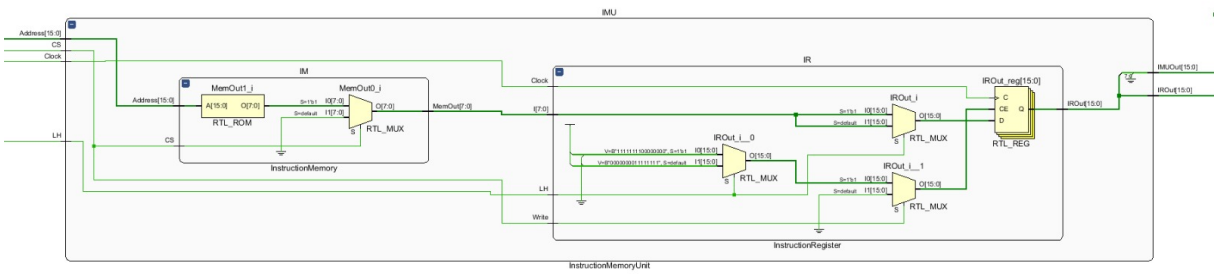

Figure 5: Inputs and Outputs of the IMU module

3.4 Part 4: ALU and System Integration

The Arithmetic Logic Unit (ALU), shown in Figure 6, performs the system's heavy computation using two 16-bit inputs (A and B) and a Carry-in (Cin). A 4-bit FunSel selects one of 16 operations, ranging from basic 2's complement addition (0100) to complex shifts like Arithmetic Shift Right (ASR) and Circular Shift (CSL/CSR).

The status of every operation is output via a 4-bit FlagsOut bus, representing Zero (Z), Carry (C), Negative (N), and Overflow (O). These flags are not volatile; they are captured into a sequential Flag Register only when the Write Flag (WF) signal is high. The Z flag is the MSB of this 4-bit register, while O is the LSB, providing a constant status update to the control unit for branching and overflow detection [4].

FunSel	**ALUOut**	**Z**	**C**	**N**	**O**
0000	A	+	-	+	-
0001	B	+	-	+	-
0010	NOT A	+	-	+	-
0011	NOT B	+	-	+	-
0100	A + B	+	+	+	+
0101	A + B + Carry	+	+	+	+
0110	A – B	+	+	+	+
0111	A AND B	+	-	+	-
1000	A OR B	+	-	+	-
1001	A XOR B	+	-	+	-
1010	A NAND B	+	-	+	-
1011	LSL A	+	+	+	-
1100	LSR A	+	+	+	-
1101	ASR A	+	-	-	-
1110	CSL A	+	+	+	-
1111	CSR A	+	+	+	-

Table 5: Characteristic table of ALU.

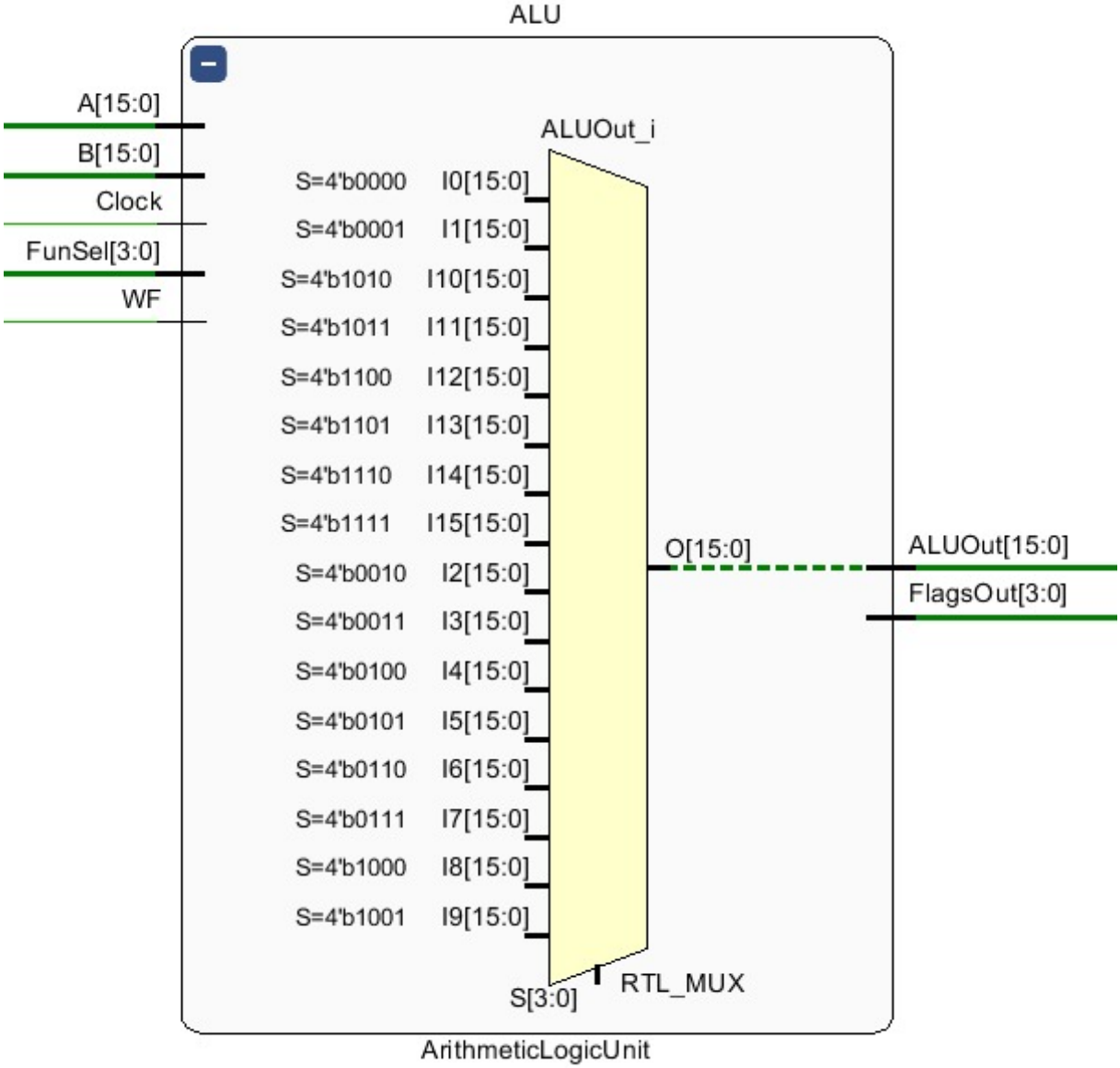

Figure 6: Inputs and Outputs of the Arithmetic Logic Unit module

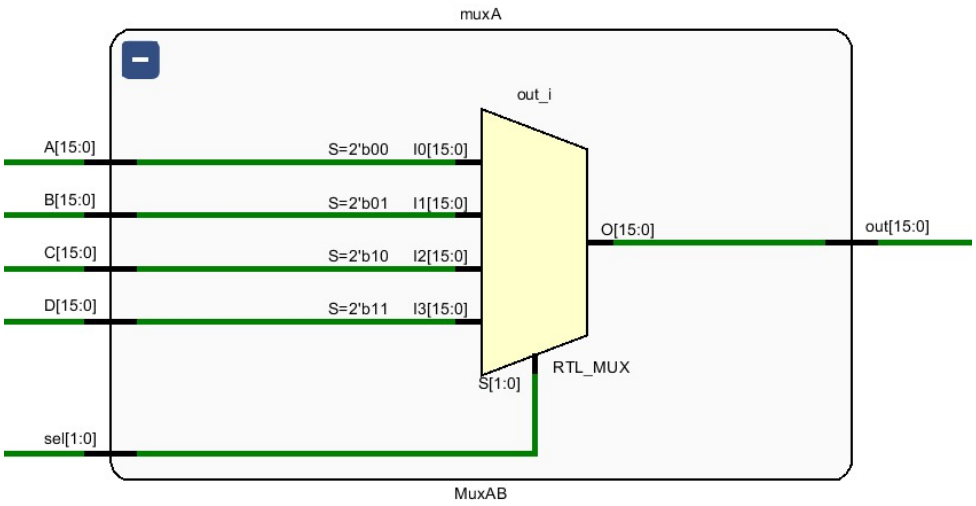

Figure 7: Inputs and Outputs of the MUX AB module

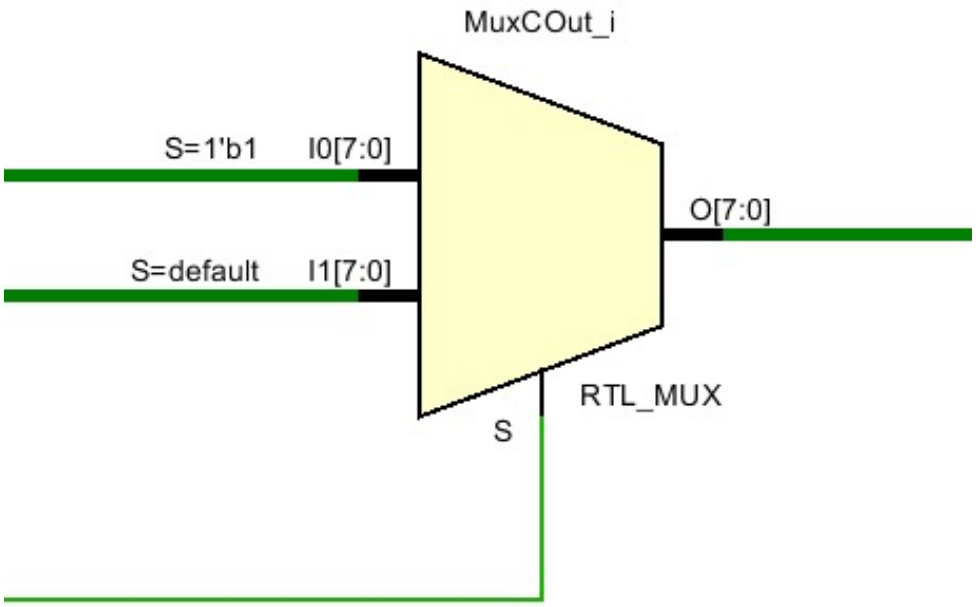

Figure 8: Inputs and Outputs of the MUX C module

4 RESULTS

Our modular implementation was rigorously tested and successfully passed all automated simulation testbenches executed via the provided `Run.bat` file in Vivado 2017.4.

By verifying each sub-module independently before top-level integration, we ensured that all control signals correctly orchestrated the datapath without timing violations or data collisions.

4.1 Register and Register File Performance

Simulation waveforms for the individual 16-bit registers confirmed that the four primary functions—Retain Value, Clear, Load, Increment, and Decrement—operated perfectly according to the 2-bit *FunSel* control inputs.

Within the Register File (RF), the complex 4-bit *RegSel* and *ScrSel* logic successfully isolated and enabled combinations of the four general-purpose (R1-R4) and four scratch (S1-S4) registers. The multiplexed outputs, *OutA* and *OutB*, correctly routed the selected 16-bit data based on the 3-bit *OutASel* and *OutBSel* signals. Similarly, the Address Register File (ARF) correctly handled the Program Counter (PC), Stack Pointer (SP), and Address Register (AR), and we verified that the dedicated *OutE* output continuously tracked the PC uninterrupted.

4.2 Memory Unit Synchronization

A critical milestone in our results was the successful validation of the 8-bit to 16-bit bus bottleneck within the memory interfaces. Simulations of the Data Memory Unit (DMU) and Instruction Memory Unit (IMU) confirmed that the 16-bit registers (DR and IR) accurately loaded data over two separate clock cycles.

In the IMU, the *L'H* control signal effectively toggled between loading the lower byte (bits 7-0) and the higher byte (bits 15-8) into the Instruction Register. In the DMU, the unified *CS=1* chip select logic allowed synchronous read/write operations between the RAM and the Data Register.

4.3 ALU and Top-Level Integration

The Arithmetic Logic Unit (ALU) outputted correct results for all 16 defined operations using 2's complement logic. This included standard arithmetic/logic functions (ADD, SUB, AND, OR, XOR, NAND) as well as complex hardware shifts (Logical, Arithmetic, and Circular shifts left and right).

To verify the "life of data" throughout the processor, a comprehensive top-level simulation was performed (see Figure 9). The logic followed a three-step sequence:

1. **Fetch:** The *IMU* was enabled to fetch a byte from ROM, which was routed via *MuxB* to update the Program Counter in the *ARF*.

2. **Execute:** Values from *R1* and *R2* were fed into the ALU. With *ALU_WF* asserted, the **Flag Register** captured the operation status, such as the Negative (N) flag for signed results.

3. **Write-Back:** The lower byte of the result was extracted via *MuxC* and stored back into the *DMU* RAM, completing the full data cycle.

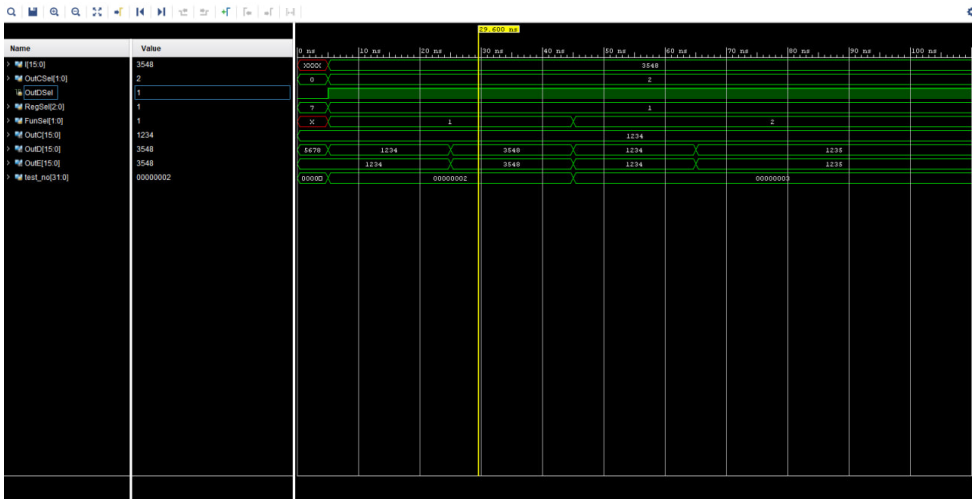

Figure 9: Top-level simulation waveforms showcasing data flow: fetching from Memory, processing in the ALU, and updating the Flag Register.

As shown in Figure 10, the synthesized connections between these modules are verified by the RTL schematic.

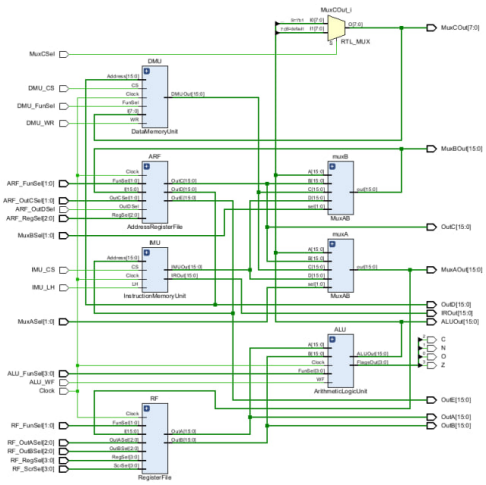

Figure 10: Vivado RTL schematics showcasing the Top-Level Module and MUX routings.

5 DISCUSSION

The implementation of the 16-bit computer organization highlighted the critical architectural distinction between combinational and sequential logic. While registers and status flags required sequential updates to maintain stable states across clock cycles, the ALU output (`ALUOut`) was transitioned to a combinational logic circuit. This design choice was essential to prevent propagation delays and timing lags within the datapath, ensuring that operands are processed immediately while the results are captured synchronously by the destination registers or the Flag Register.

Beyond timing, we successfully managed the 8-bit to 16-bit data interface by implementing multi-cycle loading logic in the DMU and IMU. The integration phase further confirmed that the 3-way multiplexer system (*MUX A, B, C*) correctly routed data between the memory units, register files, and the ALU inputs, maintaining the integrity of the data life cycle.

6 CHALLENGES AND TROUBLESHOOTING

Throughout the design and simulation phases, several logic, Verilog syntax, and documentation discrepancies were encountered and resolved.

6.1 ALU Combinational vs. Sequential Logic

During the initial ALU implementation, `ALUOut` was designed as a sequential circuit. This created synchronization issues as the rest of the datapath expected the arithmetic result to be available combinationally. We resolved this by changing the `ALUOut` assignment to combinational logic. Crucially, we maintained `FlagsOut` as a sequential circuit, which properly aligned with the system requirement that the Z, C, N, and O flags must be stored in a dedicated Flag Register.

6.2 Register File (RF) Logic and Bit Ordering

When constructing the Register File, we discovered that the output selection logic was failing because the bit ordering for the selection signals was reversed. Furthermore, we identified incorrect logic in the enablement functions for *RegSel* and *ScrSel*. By remapping the 4-bit selection lines to the structural implementation and correcting the bit order, we ensured proper synchronous updates across the eight internal registers.

6.3 Address Register File (ARF) Discrepancy

An error was identified in the project documentation regarding the *RegSel* control input for the ARF. Table 7 stated that a *RegSel* value of `011` should enable both the Stack Pointer (SP) and Address Register (AR). This conflicted with the intended operational flow. We modified the Verilog logic so that `011` strictly enabled only the Program Counter (PC), as follows:

```
3'b011:  {E_PC, E_SP, E_AR} = 3'b100;
```

6.4 Data Memory (DM) Conflict

During DMU integration, a severe conflict in the Chip Select (*cs*) logic prevented synchronous operation. Documentation specified that the RAM was active at `CS=0`, while the top-level DMU and Data Register were active at `CS=1`. This contradiction meant the RAM would be disabled exactly when the register attempted to operate. We unified the enable logic to trigger correctly at `CS=1` for both components.

6.5 Simulation Configuration: File Namings and Timescales

Compilation errors in Vivado occurred due to inconsistent module naming that did not match the project's required declarations. Additionally, missing `` `timescale `` directives caused simulation inconsistencies. Appending appropriate timescale headers and standardizing module names across all Verilog files resolved these errors.

7 CONCLUSION

During this project, we encountered several challenges regarding documentation discrepancies and timing issues. Specifically, we had to correct the *RegSel* logic in the ARF and resolve conflicting Chip Select (*cs*) logic in the DMU to ensure the RAM and Data Register operated in unison. We learned the importance of precise bit-ordering in Verilog and the necessity of timescale directives for reliable simulation. Ultimately, the project provided hands-on experience in hardware modularity and the synchronization of complex digital systems.

REFERENCES

[1] Samir Palnitkar. *Verilog HDL: A Guide to Digital Design and Synthesis*. Prentice Hall, 2nd edition, 2003.

[2] Xilinx, Inc. *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing (UG973)*, 2017.

[3] M. Morris Mano, Charles R. Kime, and Tom Martin. *Logic and Computer Design Fundamentals*. Pearson, 5th edition, 2017.

[4] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 6th edition, 2020.